08 · Generate & Stream Your First Tokens

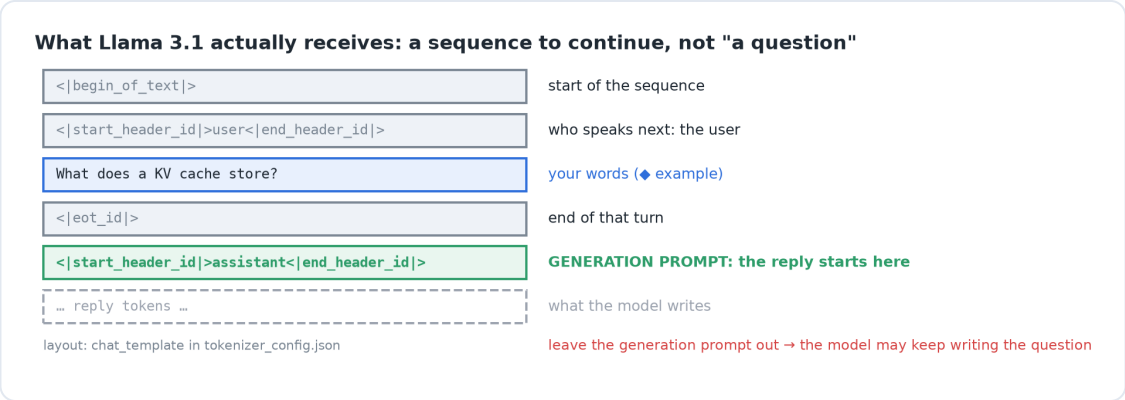
Source: fanout.sh / inference-eng / 01-03 · ~8 min

> **IN SHORT**
>
> What a user sees, **a pause then a steady stream**, is the whole of inference from the outside. The prompt is wrapped in a **chat template** and tokenized; `generate()` runs **one forward pass per token**. The first pass reads the whole prompt (**the pause, TTFT**); every later pass reads all the weights once (**the pace, TPOT**, ≥ 4.8 ms on an H100). Streaming is the **same loop with a hook**. Generation stops for exactly three reasons, and the tokens/s you see is easy to mis-measure.

Running example: **Llama 3.1 8B Instruct on one H100**.

1. What the model actually receives

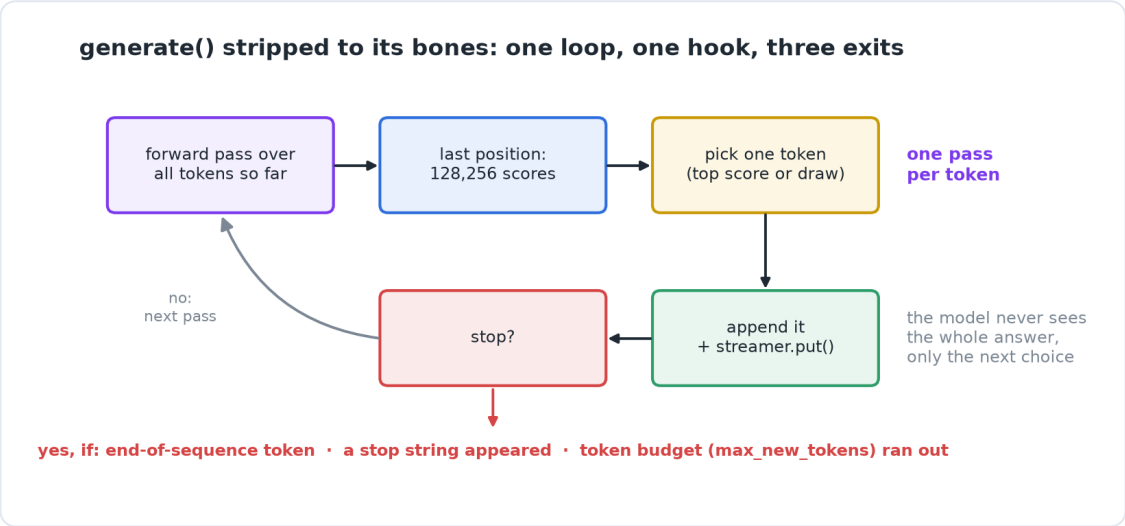
A language model only **continues a sequence of tokens**. "User" and "assistant" are conventions learned from how training data was laid out; the **chat template** (shipped in `tokenizer_config.json`) reproduces that layout.



Your message is wrapped in special markers, then the template appends an assistant header and stops.

- That final assistant header is the **generation prompt**: the cue that the next tokens are the reply. Leave it out and the model may keep writing *your* question, the most natural continuation.
- The tokenizer then turns the text into integers: a ~30-word question ≈ **~30 tokens** (illustrative).

2. The loop



Run the model, take the last position's 128,256 scores, pick one token, append it, check whether to stop, repeat.

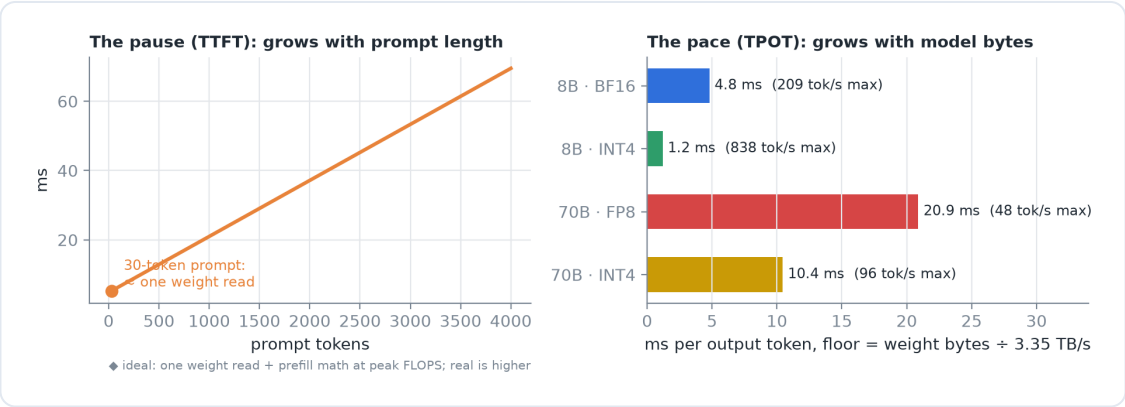
KEY IDEA

One forward pass per token. The model never sees the whole answer, only the next choice. And the first trip round the loop is different from all the others.

3. The pause and the pace

- **First pass (prefill):** reads all ~30 prompt tokens **at once**, in parallel across every layer, and stores their **keys and values** for later. Real arithmetic: the GPU is busy.
- **Every later pass (decode):** adds **one** token and reuses the stored K/V. Tiny arithmetic, but it still reads all **16 GB** of weights:

$$t_{\text{token}} \geq \frac{16\ \text{GB}}{3.35\ \text{TB/s}} \approx 4.8\ \text{ms} \quad (\approx 200\ \text{tok/s})$$



Left: a longer prompt means a longer pause. Right: a bigger (or less quantized) model means a slower pace. The loop is the same.

The pause = **time to first token (TTFT)**; the pace = **time per output token (TPOT)**.

4. Streaming: the same loop with a hook

In Transformers you pass `generate()` a **streamer**; every time a token is picked, the loop calls `streamer.put()`.

- `TextStreamer` keeps the tokens so far, decodes the whole buffer, and prints only what's new.
- It **holds back everything after the last space**: a token is often half a word, and the next token can change how the previous one should be spelled. So the display trails the model by about a word, **on purpose**; a newline flushes it.

TextStreamer trails the model by about a word, on purpose; a newline flushes it

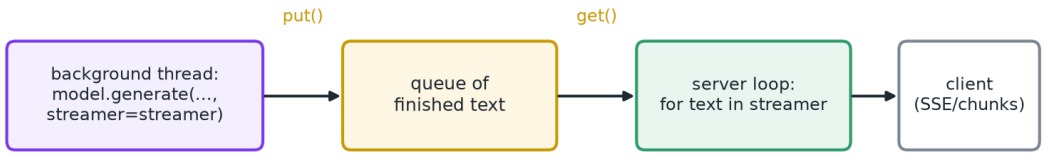
token buffer	printed so far	held back (after last space)
`Stream`	·	`Stream`
`Stream|ing`	·	`Streaming`
`Stream|ing| shows`	`Streaming`	`shows`
`…| shows| wor`	`Streaming shows`	`wor`
`…| wor|ds`	`Streaming shows`	`words`
`…|ds|\n`	`Streaming shows words\n`	`(newline: flushed)`

◆ illustrative tokenization: a token is often half a word, and the next one can change its spelling

Text is printed only up to the last space; the partial word waits for the next token.

- `TextIteratorStreamer` pushes finished text into a **queue**; your server reads from the other end without blocking the loop. That's the seam where a chat server plugs in.

TextIteratorStreamer: the same hook, feeding a queue



the generation loop never blocks on the network: this is the seam where a chat server plugs in

generate() runs in a background thread; the server iterates over the queue and forwards text to the client.

5. Why it stopped

Only three reasons, and **none of them is punctuation**:

Reason	Where it's set
The model picked an **end-of-sequence token**	`generation_config.json` : Llama 3.1 lists **three** EOS ids
A **stop string** you asked for appeared	Per request
The **token budget** ran out	`max_new_tokens` : **defaults to 20** in Transformers unless the model config says otherwise

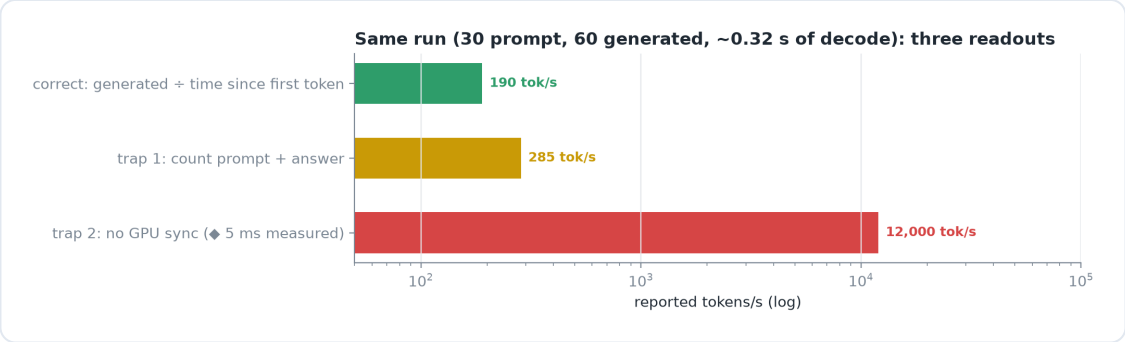
> **TRAP**
>
> An unset `max_new_tokens` is the classic cause of an answer that **ends mid-sentence**.

The same config carries this model's **sampling defaults**: sampling on, **temperature 0.6**, **top-p 0.9**. Those are someone's choices, not laws: read them, and override per request.

6. Reading the numbers

Most tools print: **prompt tokens** (what went in, after the template), **generated tokens** (what came out), and **tokens/s**, usually *generated tokens ÷ time since the first token*.

Our request (illustrative): ~**30** prompt tokens, **60** generated, a pause of a few hundred ms, then roughly **200 tok/s** at the floor (the floor is arithmetic).



The same run reported three ways: correctly, counting the prompt as output, and timing an unsynchronized GPU call.

- **Trap 1:** `generate()` returns **prompt + answer**; count the whole output and you overcount.
- **Trap 2:** GPU work is **asynchronous**: the Python call returns before the GPU finishes, so a stopwatch around it measures almost nothing. **Synchronize first**, or use **CUDA events**.

> **TRAP**
>
> - "The model is slow to start": the first step is doing ~30 tokens of work in one go.
> - "Streaming makes generation faster": it changes **when you see** tokens, not when they're made.
> - "It stops at the end of a sentence": it stops on an end token, a stop string or a budget.

7. Recap

Lesson 08 on one card

IN

chat template + tokenizer
the final assistant header
is the generation prompt

PAUSE & PACE

first pass reads the whole prompt (TTFT)
every later pass reads all weights
≥ 4.8 ms per token (TPOT)

STREAM

same loop + a hook: put() per token
holds back after the last space
iterator → queue → server

STOP & MEASURE

EOS · stop string · max_new_tokens
(default 20!) · count only generated,
sync the GPU before timing